

Sveučilište Josipa Jurja Strossmayera u Osijeku

**Fakultet elektrotehnike, računarstva i informacijskih tehnologija Osijek**

Računarstvo usluga i analiza podataka

Predložak za prvu laboratorijsku vježbu

# Dohvaćanje podataka putem REST API-ja i mrežnih stranica

---

Python / HTTP i REST / JSON, HTML i CSV

Sveučilišni diplomski studiji / DR1-13 i DA1-06

Akademska godina 2026./2027.

# Sadržaj

**1 Uvod** **1**

1.1 Rad u Google Colabu . . . 1

1.2 Biblioteke i radne datoteke . . . 1

**2 Mrežni API, HTTP i REST** **2**

2.1 API, klijent i poslužitelj . . . 2

2.2 Adresa i parametri zahtjeva . . . 2

2.3 Što sadrže HTTP poruke? . . . 3

2.4 HTTP metode . . . 3

2.5 Statusi i pogreške . . . 4

2.6 JSON odgovor i poziv iz Pythona . . . 4

2.7 REST i oblikovanje sučelja . . . 5

**3 Dohvat povijesnih vremenskih podataka** **6**

3.1 Izvor i obuhvat . . . 6

3.2 Parametri upita . . . 6

3.3 Dohvat i spremanje izvornog odgovora . . . 7

3.4 Pretvaranje JSON-a u tablicu . . . 8

3.5 Spremanje i ponovno učitavanje tablice . . . 8

**4 Izdvajanje podataka iz HTML-a** **9**

4.1 Što je web scraping? . . . 9

4.2 HTML elementi i njihovi atributi . . . 9

4.3 Selektori i odabir elemenata . . . 10

4.4 Dohvat stranice . . . 10

4.5 Izdvajanje i spremanje citata . . . 10

4.6 Straničenje . . . 11

**5 Zadaci za samostalan rad** **12**

**6 Literatura i dokumentacija** **13**

# 1. Uvod

U vježbi dohvaćamo podatke na dva načina: putem API-ja i iz HTML-a mrežne stranice. Na primjeru povijesnih vremenskih podataka i citata prolazimo dohvat, spremanje izvornog odgovora i izdvajanje zapisa za daljnju obradu. Potrebno je poznavati osnove Pythona: liste, rječnike, petlje i funkcije.

## 1.1. Rad u Google Colabu

Na vježbi koristimo **Google Colab** i priloženu bilježnicu uz ovaj LV. Colabu pristupamo kroz preglednik, a Python kod izvršava se na udaljenom računalu. Za ovu vježbu dovoljan je besplatni pristup, bez lokalne instalacije Pythona. Bilježnica sadrži ćelije za kod i tekst u formatu Markdown, a rezultati se prikazuju ispod koda. Može se otvoriti i u Jupyter Notebooku ili JupyterLabu [1].

U Colabu učitajte bilježnicu opcijom *Upload notebook* i spremite svoju kopiju. Ćelije pokrećite redom, odozgo prema dolje, tipkom za pokretanje ili kombinacijom Shift+Enter. Ćelije dijele varijable u memoriji izvršnog okruženja. Prije završnog spremanja ponovno pokrenite okruženje i izvedite sve ćelije redom.

## 1.2. Biblioteke i radne datoteke

U prvoj ćeliji instalirajte biblioteke:

```
%pip install -q requests beautifulsoup4 pandas
```

| Biblioteka | Uporaba |
|---|---|
| `requests` | Slanje HTTP zahtjeva i čitanje odgovora. |
| `beautifulsoup4` | Čitanje strukture HTML-a i pronalaženje elemenata; uvoz: `bs4`. |
| `pandas` | Rad s tablicama i spremanje CSV-a. |

Izvorne odgovore čuvamo odvojeno od podataka koje iz njih izdvojimo:

```
from pathlib import Path

RAD = Path("RUAP_LV1_podaci")
IZVORNO = RAD / "izvorno"
OBRADENO = RAD / "obradeno"
IZVORNO.mkdir(parents=True, exist_ok=True)
OBRADENO.mkdir(parents=True, exist_ok=True)
```

# 2. Mrežni API, HTTP i REST

## 2.1. API, klijent i poslužitelj

**API** (*Application Programming Interface*) je programsko sučelje: određuje kako jedan program koristi mogućnosti drugoga. Mrežni API opisuje adrese, dopuštene zahtjeve i podatke koje usluga vraća.

Program koji šalje zahtjev jest **klijent**, a program koji ga prima i obrađuje **poslužitelj**. U vježbi je klijent naš Python kod. **Resurs** je ono čemu pristupamo, primjerice jedan zadatak ili zbirka zadataka. Podatke o resursu primamo kao **reprezentaciju**, primjerice dokument u formatu JSON.

**HTTP** (*Hypertext Transfer Protocol*) je aplikacijski protokol za razmjenu zahtjeva i odgovora. **HTTPS** štiti HTTP prijenos TLS-om (*Transport Layer Security*), koji omogućuje šifriranje i provjeru identiteta poslužitelja [2]. HTTP može prenositi JSON, HTML, slike i druge vrste sadržaja.

## 2.2. Adresa i parametri zahtjeva

**URL** (*Uniform Resource Locator*) određuje adresu kojoj pristupamo. Za prvi primjer koristimo JSONPlaceholder, ogledni API za vježbanje [4]:

```
https://jsonplaceholder.typicode.com/todos?userId=1&id=1
```

| Dio URL-a | Značenje |
|---|---|
| `https` | Shema koja određuje uporabu HTTPS-a. |
| `jsonplaceholder.typicode.com` | Naziv odredišne usluge. |
| `/todos` | Putanja do zbirke zadataka. |
| `?userId=1&id=1` | Upitni parametri za odabir podataka. |

Prvi parametar slijedi nakon `?`, a ostali se odvajaju znakom `&`. Značenje parametara propisuje API: ovdje `userId=1` odabire zadatke prvog korisnika, a `id=1` zadatak s oznakom 1.

**Pristupna točka** (*endpoint*) je adresa API-ja na kojoj se izvodi podržana operacija. Primjerice, GET na `/todos/1` dohvaća pojedinačni zadatak. GET na `/todos?userId=1&id=1` filtrira zbirku. Prvi zahtjev vraća objekt, a drugi listu s jednim objektom.

## 2.3. Što sadrže HTTP poruke?

Zahtjev određuje **metodu**, odnosno radnju koju tražimo, i cilj zahtjeva. **Zaglavlja** prenose dodatne podatke, primjerice željeni format odgovora. Zahtjev može imati i **tijelo**, u kojem šaljemo sadržaj poslužitelju.

Skraćeni prikaz HTTP/1.1 zahtjeva [2]:

```
GET /todos?userId=1&id=1 HTTP/1.1
Host: jsonplaceholder.typicode.com
Accept: application/json
```

Metoda `GET` traži dohvat sadržaja. Zaglavlje `Host` navodi odredište, a `Accept` željeni format odgovora. Prazan redak odvaja zaglavlja od tijela; u ovom zahtjevu tijela nema.

Odgovor sadrži **status**, zaglavlja i, kada je primjenjivo, tijelo:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

[{"userId": 1, "id": 1, "title": "...", "completed": false}]
```

Tekst naslova u primjeru skraćen je. Status `200` znači da je zahtjev uspio. `Content-Type` opisuje format tijela odgovora. Isto zaglavlje u zahtjevu opisivalo bi format sadržaja koji šaljemo. Poslužitelj ipak ne mora vratiti format naveden u zaglavlju `Accept`.

## 2.4. HTTP metode

Metoda određuje što želimo učiniti s resursom. API na nekoj adresi može podržavati samo dio metoda [2].

| Metoda | Namjena |
|---|---|
| GET | Dohvat reprezentacije resursa. |
| HEAD | Kao GET, ali bez tijela odgovora. |
| POST | Slanje sadržaja na obradu; često stvaranje novog zapisa. |
| PUT | Stvaranje ili potpuna zamjena stanja resursa na zadanoj adresi. |
| PATCH | Djelomična izmjena prema podržanom formatu izmjene. |
| DELETE | Zahtjev za uklanjanje resursa na zadanoj adresi. |

**Sigurna metoda** ne traži promjenu stanja resursa. GET i HEAD sigurni su, pri čemu je bilježenje pristupa u dnevnik i dalje dopušteno. **Idempotentna metoda** pri ponavljanju istog zahtjeva ima isti namjeravani učinak kao jedan zahtjev. GET, HEAD, PUT i DELETE idempotentni su; POST i PATCH to općenito nisu [2]. Idempotentnost GET-a ne znači da se njegov odgovor ne može promijeniti ako se promijeni izvor.

## 2.5. Statusi i pogreške

Statusi se dijele u skupine: `1xx` informativni odgovori, `2xx` uspjeh, `3xx` preusmjeravanje i povezani ishodi, `4xx` pogreške zahtjeva te `5xx` pogreške poslužitelja [2].

| Status | Značenje |
|---|---|
| 200 OK | Zahtjev je uspio. |
| 204 No Content | Uspješan odgovor bez tijela. |
| 301 / 302 | Preusmjeravanje na drugu adresu. |
| 400 Bad Request | Neispravan zahtjev. |
| 401 / 403 | Nedostaju valjane vjerodajnice / pristup je odbijen. |
| 404 Not Found | Traženi resurs nije pronađen. |
| 429 Too Many Requests | Prekoračeno je ograničenje broja zahtjeva. |
| 500 / 503 | Interna pogreška / usluga je nedostupna. |

Requests uobičajeno prati preusmjeravanja GET zahtjeva. Poziv `raise_for_status()` za statuse 4xx i 5xx podiže iznimku `requests.HTTPError`. Problemi s vezom mogu izazvati iznimku `requests.ConnectionError`, a istek čekanja `requests.Timeout` [3].

Argument `timeout=30` ograničava čekanje na povezivanje i na podatke pri čitanju na 30 sekundi; ne određuje ukupno trajanje zahtjeva. Poruka o pogrešci također može biti JSON, pa uspjeh zahtjeva provjeravamo prema statusu, prije čitanja podataka.

Kod statusa 400 ispravite zahtjev prema poruci API-ja. Kod 429 pričekajte; `Retry-After`, ako je prisutan, navodi odgodu u sekundama ili HTTP datum.

## 2.6. JSON odgovor i poziv iz Pythona

**JSON** (*JavaScript Object Notation*) tekstualni je format podataka. Objekt povezuje nazive polja i njihove vrijednosti, a lista (*array*) sadrži uređeni niz vrijednosti. Vrijednosti mogu biti tekst, broj, `true`, `false`, `null`, drugi objekt ili lista.

Poziv `response.json()` pretvara JSON u Python strukture [3]:

| JSON | Python |
|---|---|
| Objekt | Rječnik, `dict`. |
| Lista (array) | Lista, `list`. |
| Tekst, broj | `str`, `int` ili `float`. |
| `true`, `false`, `null` | `True`, `False`, `None`. |

Requests šalje zahtjev i vraća objekt odgovora. Parametre predajemo kao rječnik `params`, a biblioteka ih kodira u URL-u [3].

```
import requests

http_odgovor = requests.get(
    "https://jsonplaceholder.typicode.com/todos",
    params={"userId": 1, "id": 1},
    headers={"Accept": "application/json"},
    timeout=30,
)
print(http_odgovor.url)
print(http_odgovor.status_code)
print(http_odgovor.headers.get("Content-Type"))
http_odgovor.raise_for_status()
http_podaci = http_odgovor.json()
print(type(http_podaci).__name__, len(http_podaci))
```

## 2.7. REST i oblikovanje sučelja

**REST** (*Representational State Transfer*) arhitekturni je stil. Određuje kako se oblikuju sučelja i razmjenjuju reprezentacije resursa u raspodijeljenom sustavu.

Fielding definira sljedeća ograničenja [2]:

**Klijent i poslužitelj.** Razdvojene odgovornosti omogućuju neovisan razvoj.

**Bez sesijskog stanja (stateless).** Svaki zahtjev sadrži podatke potrebne za obradu i ne oslanja se na kontekst ranijih zahtjeva. Poslužitelj pritom može čuvati trajne podatke o resursima.

**Predmemoriranje.** Odgovor određuje smije li se ponovno uporabiti.

**Jedinstveno sučelje.** Resursi imaju identifikatore, pristupa im se preko reprezentacija, poruke opisuju vlastito značenje, a poveznice vode prema dostupnim sljedećim radnjama (hipermedija).

**Slojevi.** Između klijenta i odredišne usluge mogu biti posrednici.

**Kod na zahtjev.** Poslužitelj može poslati izvršivi kod; ovo je neobavezno.

U praksi se naziv REST API često koristi za HTTP sučelja organizirana oko resursa, iako neka od njih ne zadovoljavaju sva REST ograničenja. Primjerice, klijent katkad mora unaprijed znati adrese dostupnih operacija umjesto da ih pronalazi iz poveznica u odgovorima. HTTP određuje način razmjene zahtjeva i odgovora, dok REST opisuje kako je sučelje oblikovano. JSON je jedan od mogućih formata podataka koje takvo sučelje prenosi.

# 3. Dohvat povijesnih vremenskih podataka

## 3.1. Izvor i obuhvat

Open-Meteo Historical Weather API omogućuje dohvat povijesnih vremenskih podataka bez API ključa. Odabiremo ERA5, reanalizu koja kombinira opažanja i meteorološki model. Vrijednosti se odnose na točku prostorne mreže blizu tražene lokacije, a ne na očitanja jedne meteorološke postaje [5].

Besplatni API pristup namijenjen je nekomercijalnoj uporabi, uključujući nastavu, uz ograničenja broja zahtjeva. Podaci su dostupni pod licencom CC BY 4.0: navedite izvor, poveznicu na licencu i napravljene promjene [5].

Dohvaćamo podatke za razdoblje **od 1. do 7. siječnja 2024., uključivo**, u UTC-u. **UTC** (*Coordinated Universal Time*) zajednička je vremenska referenca bez sezonskog pomicanja sata. U tablici koju ćemo izraditi jedan redak predstavlja **jedan grad i jedan sat**.

| Lokacija | Geografska širina | Geografska dužina |
|---|---|---|
| Osijek | 45.55 | 18.70 |
| Zagreb | 45.81 | 15.98 |
| Split | 43.51 | 16.44 |

## 3.2. Parametri upita

Zahtjev oblikujemo parametrima iz dokumentacije API-ja [5]:

| Parametar | Odabir |
|---|---|
| `latitude`, `longitude` | Koordinate tražene lokacije u decimalnim stupnjevima. |
| `start_date`, `end_date` | Prvi i posljednji uključeni datum. |
| `hourly` | Popis satnih varijabli odvojenih zarezima. |
| `timezone` | `UTC`. |
| `models` | `era5`, izričito odabrani model reanalize. |

```
api_url = "https://archive-api.open-meteo.com/v1/archive"
api_params = {
    "latitude": 45.55, "longitude": 18.70,
    "start_date": "2024-01-01", "end_date": "2024-01-07",
    "hourly": "temperature_2m,relative_humidity_2m,precipitation",
    "timezone": "UTC",
    "models": "era5",
}
```

### 3.3. Dohvat i spremanje izvornog odgovora

Pošaljite zahtjev s prethodno definiranim parametrima:

```
from datetime import datetime, timezone

api_odgovor = requests.get(
    api_url, params=api_params, timeout=30
)
api_preuzeto_utc = datetime.now(timezone.utc).isoformat()
api_odgovor.raise_for_status()
api_podaci = api_odgovor.json()
print(api_podaci.keys())
print(api_podaci["hourly_units"])
```

Odgovor je JSON objekt. `hourly` sadrži liste satnih vrijednosti, `hourly_units` njihove jedinice, a polja `latitude` i `longitude` vraćene koordinate. Prije izrade tablice spremite tijelo odgovora i **metapodatke**: opis odakle je sadržaj preuzet i kojim zahtjevom.

```
import json

def spremi_odgovor(odgovor, putanja, preuzeto_utc):
    meta = {
        "retrieved_at_utc": preuzeto_utc,
        "request_url": odgovor.url,
    }
    putanja.write_bytes(odgovor.content)
    meta_putanja = putanja.with_suffix(".meta.json")
    meta_putanja.write_text(
        json.dumps(meta, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

api_putanja = IZVORNO / "osijek.json"
spremi_odgovor(api_odgovor, api_putanja, api_preuzeto_utc)
```

Funkciju koristimo i za spremanje HTML-a u sljedećem poglavlju. Za svaki upit odaberite zaseban naziv datoteke.

Izvorni JSON možemo ponovno pročitati bez novog mrežnog zahtjeva:

```
spremljeni_podaci = json.loads(api_putanja.read_text(encoding="utf-8"))
print(len(spremljeni_podaci["hourly"]["time"]))
```

### 3.4. Pretvaranje JSON-a u tablicu

Objekt `hourly` sadrži listu vremena i liste odabranih varijabli. Elementi na istom položaju pripadaju istom retku. Skraćeni primjer s oglednim vrijednostima:

```
{"hourly": {
    "time": ["2024-01-01T00:00", "2024-01-01T01:00"],
    "temperature_2m": [8.0, 10.0]
}}
```

Iz jednako dugih lista nastaje `DataFrame`, pandasova tablica:

```
import pandas as pd

df_osijek = pd.DataFrame(spremljeni_podaci["hourly"])
df_osijek = df_osijek.rename(columns={"time": "time_utc"})
df_osijek["time_utc"] = pd.to_datetime(df_osijek["time_utc"], utc=True)
df_osijek.insert(0, "location", "Osijek")
print(df_osijek.head())
print(df_osijek.dtypes)
```

`utc=True` pridružuje UTC vremenskim zapisima jer smo tu zonu zatražili u upitu [3].

| Polje | Značenje i jedinica |
|---|---|
| `location` | Naziv traženog grada. |
| `time_utc` | Vremenski zapis u UTC-u. |
| `temperature_2m` | Temperatura zraka na 2 m, u °C, za označeni trenutak. |
| `relative_humidity_2m` | Relativna vlažnost na 2 m, u %. |
| `precipitation` | Količina oborine u mm za prethodni sat. |

### 3.5. Spremanje i ponovno učitavanje tablice

**CSV** (*Comma-Separated Values*) tekstualni je tablični format. U izvozu koristimo zarez kao razdjelnik i točku kao decimalni znak [3].

```
csv_putanja = OBRADENO / "osijek.csv"
df_osijek.to_csv(csv_putanja, index=False, encoding="utf-8")

ponovno = pd.read_csv(csv_putanja)
ponovno["time_utc"] = pd.to_datetime(ponovno["time_utc"], utc=True)
print(len(ponovno), ponovno.columns.tolist())
```

CSV ne čuva vremenski tip stupca, pa ga nakon čitanja ponovno pretvaramo. Opis polja i jedinica, poput prethodne tablice, nazivamo **rječnikom podataka**.

# 4. Izdvajanje podataka iz HTML-a

## 4.1. Što je web scraping?

**Web scraping** izdvaja podatke iz mrežne stranice. HTML je prilagođen prikazu sadržaja, pa promjena strukture stranice može prekinuti izdvajanje. Dokumentirani API ili izravno preuzimanje skupa, kada postoje, obično su jednostavniji izbor.

**Crawling** je otkrivanje i obilaženje adresa praćenjem poveznica. U vježbi ćemo pratiti poveznicu na sljedeću stranicu i izdvojiti citate s prvih dviju stranica. Koristimo https://quotes.toscrape.com/, dio projekta *Web Scraping Sandbox*, namijenjenog učenju. Na osnovnoj su stranici citati, autori, oznake i poveznica na sljedeću stranicu [6].

Prije dohvata provjerite uvjete uporabe i `robots.txt`, koji daje upute automatiziranim klijentima za pristup putanjama, ali ne određuje licencu sadržaja. Uz prikupljeni skup navedite izvor i dostupne uvjete uporabe; ako licenca tekstova nije navedena, to zabilježite [6].

## 4.2. HTML elementi i njihovi atributi

**HTML** (*HyperText Markup Language*) opisuje strukturu dokumenta. Elementi su zadani oznakama kao što su `div` i `span`, mogu imati atribute te sadržavati tekst ili druge elemente. Sljedeći primjer prikazuje strukturu jednog citata s oglednim tekstom:

```
<div class="quote">
  <span class="text">Primjer kratkog citata.</span>
  <small class="author">Ime Autora</small>
  <div class="tags">
    <a class="tag" href="/tag/ucenje">ucenje</a>
  </div>
</div>
```

Atribut `class` označuje razrede elemenata, a `href` adresu poveznice. Više elemenata može pripadati istom razredu.

**Parser** čita HTML i gradi stablo elemenata koje možemo pretraživati. Beautiful Soup omogućuje rad s tim stablom. Sam ne preuzima stranice i ne izvršava JavaScript; HTML mu predajemo nakon dohvata bibliotekom Requests. Ako se sadržaj dodaje JavaScriptom, možda ga neće biti u dohvaćenom HTML-u. Citati s odabrane stranice dostupni su u izvornom HTML-u.

### 4.3. Selektori i odabir elemenata

**CSS** (*Cascading Style Sheets*) služi oblikovanju mrežnih stranica. CSS selektor opisuje koje elemente odabiremo. Beautiful Soup podržava njihovu uporabu za pretraživanje [6].

| Selektor | Odabir |
|---|---|
| `div.quote` | Elementi `div` s razredom `quote`. |
| `span.text` | Elementi `span` s razredom `text`. |
| `small.author` | Elementi `small` s razredom `author`. |
| `.tags a.tag` | Poveznice razreda `tag` unutar elementa razreda `tags`. |
| `li.next a` | Poveznice unutar elementa `li` s razredom `next`. |

`select()` vraća sve podudarne elemente, a `select_one()` prvi ili `None`. Metoda `get_text()` čita tekst elementa, a `element["href"]` čita vrijednost atributa `href`.

Strukturu istražite opcijom *Inspect/Elements* u pregledniku, a izvorni odgovor pregledajte u kartici *Network*.

### 4.4. Dohvat stranice

Najprije dohvatite i spremite HTML. Koristimo funkciju `spremi_odgovor()` iz odjeljka 3.3:

```
html_url = "https://quotes.toscrape.com/"
html_odgovor = requests.get(html_url, timeout=30)
html_preuzeto_utc = datetime.now(timezone.utc).isoformat()
html_odgovor.raise_for_status()
spremi_odgovor(html_odgovor, IZVORNO / "quotes_page_1.html", html_preuzeto_utc)
```

### 4.5. Izdvajanje i spremanje citata

Dohvaćeni HTML predajemo Beautiful Soupu koristeći parser `html.parser`.

```
from bs4 import BeautifulSoup
from urllib.parse import urljoin

soup = BeautifulSoup(html_odgovor.content, "html.parser")
blokovi = soup.select("div.quote")
if not blokovi:
    raise ValueError("Na stranici nisu đpronaeni citati.")
print("Broj blokova:", len(blokovi))
```

Za svaki blok zajedno izdvajamo tekst, autora i oznake:

```
citati = []
for blok in blokovi:
    citati.append({
        "text": blok.select_one("span.text").get_text(" ", strip=True),
        "author": blok.select_one("small.author").get_text(" ", strip=True),
        "tags": [x.get_text(strip=True)
                 for x in blok.select(".tags a.tag")],
        "source_url": html_odgovor.url,
    })
```

Zapise spremamo u JSON kako bi oznake ostale lista:

```
(OBRADENO / "quotes_page_1.json").write_text(
    json.dumps(citati, ensure_ascii=False, indent=2), encoding="utf-8"
)
```

## 4.6. Straničenje

**Straničenje** (*pagination*) dijeli skup na više stranica. API-ji za to mogu koristiti broj stranice, pomak ili kursor. Na odabranoj mrežnoj stranici slijedimo poveznicu *Next*:

```
dalje = soup.select_one("li.next a")
sljedeci_url = urljoin(html_odgovor.url, dalje["href"]) if dalje else None
print(sljedeci_url)
```

`urljoin()` pretvara relativni `href` u potpunu adresu prema URL-u trenutačnog dokumenta. Ako poveznice nema, `sljedeci_url` ostaje `None`, pa obilazak završava.

# 5. Zadaci za samostalan rad

Rješenja i kratka objašnjenja unesite u priloženu bilježnicu.

### Zadatak 1. HTTP zahtjevi i odgovori

a) Usporedite GET zahtjeve na `/todos?userId=1&id=1` i `/todos/1` s JSONPlaceholdera. Ispišite status, `Content-Type` i tijelo odgovora te objasnite razliku u obliku podataka.

b) Zatražite nepostojeći zadatak `/todos/999999` i obradite nastalu iznimku. Objasnite zašto provjeravamo status prije čitanja JSON-a.

c) Dohvatite sve zadatke korisnika 2, a zatim samo njegove dovršene zadatke. Parametre predajte argumentom `params`; za `completed` upotrijebite tekst `"true"`. Usporedite broj zapisa i provjerite pripadaju li vraćeni zadaci tom korisniku.

d) Za `/todos/1` pošaljite i HEAD zahtjev pozivom `requests.head()`. Usporedite status, zaglavlja i tijelo s GET odgovorom te objasnite uočenu razliku.

### Zadatak 2. Dohvat vremenskih podataka

a) Proširite primjer na Osijek, Zagreb i Split. Koristite koordinate iz odjeljka 3.1 te iste varijable, razdoblje i ostale parametre upita. Izvorni odgovor i metapodatke spremite zasebno za svaki grad.

b) Spojite podatke svih gradova u jednu tablicu sa stupcima iz odjeljka 3.4; možete koristiti `pd.concat()`. Pregledajte broj redaka i nekoliko zapisa, a zatim spremite tablicu kao `vrijeme.csv`. Ponovno učitajte datoteku i provjerite jesu li sačuvani podaci i vremenske oznake.

c) Proširite upit varijablom `wind_speed_10m`, ponovno dohvatite podatke za sva tri grada i dodajte brzinu vjetra u zajedničku tablicu. Spremite dopunjenu tablicu u CSV. Iz polja `hourly_units` pročitajte jedinicu i dopunite opis podataka.

### Zadatak 3. Izdvajanje citata

a) Dohvatite prve dvije stranice Quotes to Scrape prateći poveznicu *Next*. Za relativne adrese koristite `urljoin()`. Spremite HTML i metapodatke svake stranice; zaustavite obilazak nakon dviju stranica ili ako sljedeće poveznice nema.

b) Za svaki citat izdvojite tekst, autora, oznake i adresu izvora. Sve zapise spremite u `quotes.json`. Ispišite broj citata i nekoliko zapisa te ih usporedite sa sadržajem stranice.

c) Odaberite jednu od prikupljenih oznaka, izdvojite sve citate s tom oznakom i spremite ih u zasebnu JSON datoteku.

Uz oba skupa u Markdown ćelijama navedite izvor, opis polja i uvjete uporabe; za vremenske podatke navedite i jedinice. Provjerite može li se izdvajanje podataka ponoviti iz spremljenih odgovora, bez novog dohvata.

Bilježnicu (`.ipynb`) i arhivu s podacima (npr. ZIP ili 7z) možete predati na Merlinu. Na vježbi je potrebno nastavniku pokazati rješenje i objasniti kako radi.

## 6. Literatura i dokumentacija

[1] Radna okruženja: Google Colab i Project Jupyter.

[2] HTTP i REST: HTTP Working Group: standardi; R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, 2000.

[3] Dokumentacija biblioteka: Requests, Beautiful Soup i pandas.

[4] typicode. JSONPlaceholder.

[5] Open-Meteo: API-ji, dokumentacija i uvjeti uporabe podataka.

[6] Web Scraping Sandbox: stranice za vježbanje izdvajanja podataka.